

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 1

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name : Deep Learning

Problem Statement:

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset.

Algorithm:

- Import the required TensorFlow and Matplotlib libraries.
- Load the Fashion MNIST dataset into training and testing sets.
- Normalize the image pixel values from 0–255 to 0–1.
- Display the shape of the training and testing datasets.
- Visualize a few sample images from the training dataset.
- Display a single Fashion MNIST image with its corresponding label.
- Create a Sequential neural network with a Flatten layer, one hidden Dense layer (ReLU), and an output Dense layer (Softmax).
- Compile the model using the Adam optimizer, Sparse Categorical Crossentropy loss function, and Accuracy metric.
- Train the model on the training dataset for 3 epochs.
- Evaluate the trained model on the test dataset and display the test loss and accuracy.

Code:

```
1] # import libraries

import tensorflow as tf

import matplotlib.pyplot as plt

2] # Load Fashion MNIST Dataset

(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()
```

```
3] # Normalize the Data

train_images = train_images / 255.0

test_images = test_images / 255.0

4] # Display Dataset Information

print("Training Images Shape:", train_images.shape)

print("Testing Images Shape:", test_images.shape)

print("Training Labels Shape:", train_labels.shape)

print("Testing Labels Shape:", test_labels.shape)

5] # Visualize Sample Images

plt.figure(figsize=(6,6))

for i in range(9):

    plt.subplot(3,3,i+1)

    plt.imshow(train_images[i], cmap="gray")

    plt.title(class_names[train_labels[i]])

    plt.axis("off")

plt.tight_layout()

plt.show()

6] # Display One Image

plt.imshow(train_images[7], cmap="gray")

plt.title(class_names[train_labels[7]])

plt.axis("off")

plt.show()

7] # Define Neural Network Model

my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
```

```
my_model.add(tf.keras.layers.Dense(128, activation="relu"))

my_model.add(tf.keras.layers.Dense(10, activation="softmax"))

8] # Compile the Model

my_model.compile(

    optimizer="adam",

    loss="sparse_categorical_crossentropy",

    metrics=["accuracy"]

)

9] # Train the Model

history = my_model.fit(

    train_images,

    train_labels,

    epochs=3

)

10] # Evaluate the Model

test_loss, test_accuracy = my_model.evaluate(test_images, test_labels)

print("Test Loss:", test_loss)

print("Test Accuracy:", test_accuracy)
```

Output:

1]

```
[ ] # import libraries

import tensorflow as tf
import matplotlib.pyplot as plt

[ ] # Load Fashion MNIST Dataset
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 0s 0us/step
```

2]

```
[ ] # Normalize the Data
train_images = train_images / 255.0
test_images = test_images / 255.0

[ ] # Display Dataset Information
print("Training Images Shape:", train_images.shape)
print("Testing Images Shape:", test_images.shape)

print("Training Labels Shape:", train_labels.shape)
print("Testing Labels Shape:", test_labels.shape)

... Training Images Shape: (60000, 28, 28)
Testing Images Shape: (10000, 28, 28)
Training Labels Shape: (60000,)
Testing Labels Shape: (10000,)
```

3]



4]



5]

```
# Define Neural Network Model
my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation="relu"))
my_model.add(tf.keras.layers.Dense(10, activation="softmax"))

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(**kwargs)

# compile the model
my_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

6]

```
# Train the Model
history = my_model.fit(
    train_images,
    train_labels,
    epochs=3
)
```

*** Epoch 1/3
1875/1875 ————— 9s 5ms/step - accuracy: 0.8245 - loss: 0.4996
Epoch 2/3
1875/1875 ————— 9s 4ms/step - accuracy: 0.8642 - loss: 0.3738
Epoch 3/3
1875/1875 ————— 7s 4ms/step - accuracy: 0.8785 - loss: 0.3328

7]

```
# Evaluate the Model
test_loss, test_accuracy = my_model.evaluate(test_images, test_labels)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

*** 313/313 ————— 1s 2ms/step - accuracy: 0.8623 - loss: 0.3792
Test Loss: 0.3792435824871063
Test Accuracy: 0.8622999787330627